

Data Poisoning of an Information Security Machine Learning Model Final Report

Spring 2022

Team Members:

Rishi Dandu - rkd160030@utdallas.edu

Ravi Teja Avuthu - rxa210038@utdallas.edu

Harrison Hamric - hgh160130@utdallas.edu

Payton Harmon - pnh170000@utdallas.edu

Ashrafi Akbar - khandakarashrafi.akbar@utdallas.edu

Project Mentors:

Olivia Brown - olivia.brown@ll.mit.edu Dr. Jason Matterer - jason.matterer@ll.mit.edu

Summary

Data poisoning is a form of a supply chain attack in which bad actors attempt to manipulate machine learning models that detect attacks and anomalies by “poisoning” the data they train on.

We believe with experimentation and research, we could attempt to test an existing open source dataset and model against existing data poisoning techniques and to help inform the maintainers of the dangers that such techniques may face and how to mitigate them.

Keywords: Data Poisoning, Machine Learning and Cyber Security Models, Improving accuracy of the models, identifying data anomalies.

Brief Project Description: Research ways to detect “quality” of datasets by for example, training versions of models simultaneously on new (poisoned) and old (clean) datasets and then test on the opposite dataset to see if there is an unexplained boost in performance in an old model on a new dataset in a particular way or a drop in performance on a new model on old data.

Furthermore, we could experiment with trying data poisoning techniques to see if they can circumvent this detection.

Assignment of Responsibilities: We as a team would learn the basics of how to poison a basic data set and reduce the accuracy of a model. Then two of us will work to identify what kind of parameters generally are targeted in such attacks and try different ML models using different data sets from Kaggle. The rest of the team will try to develop a method to identify such anomalies and increase the accuracy of the model.

Motivation

As more and more ML models become deployed to the public more and more data poisoning attacks may occur especially if the models are making important decisions. Also, there might be human oversight in the future so we need to develop methods to counteract this threat.

Specific Aims

Identify feasible techniques to prevent data poisoning on the drebin dataset and come up with general paradigms to help prevent data poisoning attacks. As more and more ML models are making decisions, the stakes of these decisions will increase. We need to identify ways the poisoning attacks are done and come with feasible methods to stop them.

We know we can accomplish this if we can see that attacks are being identified as data poisoning and mitigated. We can solve this by identifying shortcomings in our dataset and lowering accuracy of results when we classify data. We can also identify vulnerable parameters to data poisoning and be on the lookout for potential attacks.

This will build us toward a larger goal because automated ML algorithms make a lot of important decisions in people's lives. If data can be poisoned affecting outcomes, it can lead to a quality-of-life loss for Americans and everyone around the world.

We know we will be successful if we are able to successfully poison data and also come up with feasible ways to stop some of the known poisoning attacks. We can evaluate our progress to see the complexity of attacks we are able to stop.

Plan

First, we will get a basic understanding of data poisoning techniques and fundamentals and apply to a popular kaggle dataset. Then we will understand the parameters of the drebin dataset and identify weaknesses and shortcomings in the dataset and try to poison the data. Then we will take our learning and try to prevent known types of poisoning attacks.

Schedule

Phase 1 (2 weeks): Select a data set from Kaggle, understand the data set and its parameters and why they are used. Identify the parameters which can be poisoned. Try to poison the data and reduce the accuracy of the selected model which is using the data set.

Phase 2 (4 Weeks): Understand the parameters in Drebin dataset and Identify the parameters which can be poisoned. Then poison data with said parameters and reduce accuracy of the model.

Phase 3 (2 weeks): Summarize research and insight into digestible powerpoint and do knowledge transfer to mentors about said findings.

Team Approach and Justification

Technical Approach

The goal of our project to detect “quality” of datasets by for example, training versions of models simultaneously on new (poisoned) and old (clean) datasets and then test on the opposite dataset to see if there is an unexplained boost in performance in an old model on a new dataset in a particular way or a drop in performance on a new model on old data.

To accomplish our goal, we planned to start learning different data sets and different data poisoning techniques. We also had in our list how to identify the most important features and try to poison them such that the accuracies get down. The data set we selected is the drebin dataset. Drebin is one of the most used academic datasets of android malware in the information security community. We plan on using this dataset and apply the techniques we learnt both (evasion and poisoning) to get the accuracy down. For that we need to understand the importance of each parameter of that dataset.

Once we understood the model and investigated the android drebin dataset, we found the pre-existing data poisoning libraries (SecML) that we applied to the dataset. We used SVM (Support Vector Machine) as our classifier with a linear kernel. SVM is a supervised learning model that analyzes data for classification and regression analysis. It is good for supervised learning and leads to good accuracy of 89%. We generated adversarial examples to confuse SVM classifier. Poisoned the general data effectively to reduce the accuracy.

Broader Impact

The impact of identifying the impact of poisoning the data on real world malicious software datasets has broad implications for the future of malicious software Machine Learning models. The explicit impact of this project is a clarified understanding of whether adversaries can affect the accuracy of Machine Learning models that identify malicious software. The implicit impact is that it will affect the decision making of those who design Machine Learning models that identify malicious software. They will consider where they intake their data and consider whether they can ensure it has not been poisoned. This project will advance the security of malicious software detection that uses Machine Learning models. This is because it is assumed that there are no adversaries that can affect the data in this way, or it is assumed this attack has no impact. If that assumption is changed, we will have advanced the security of malicious software detection using Machine Learning models. This method of detecting malicious software is considered state-of-the-art, and thus we believe this project will enhance that. In terms of society and the project impact, we believe that malicious software affects huge amounts of end users every day. The detection and improvement of detecting that, thus, has a massive impact on society at large.

Mid Term Progress Update

First, we searched for general knowledge about data poisoning. We researched machine learning models and general data poisoning techniques. We have developed an understanding of Data Evasion vs Data Poisoning and the various related issues along with SHAP values and explainable AI fundamentals. We have acquired and investigated the Android drebin dataset. We have found pre-existing data poisoning libraries (SecML) that are applying to the Android(drebin) dataset. We have generated adversarial examples to confuse SVM classifiers (Data Evasion). Until this phase we are in the process of finding effective features to poison to reduce accuracy of classifier.

```
Explanations for sample 138 (true class: 1)
15.18 suspicious_calls::android/telephony/TelephonyManager;->getNetworkOperator
12.55 req_permissions::android.permission.SEND_SMS
8.69 req_permissions::android.permission.READ_SMS
5.83 req_permissions::android.permission.INTERNET
5.58 intent_filters::android.intent.action.BOOT_COMPLETED
-4.41 used_permissions::VIBRATE
3.96 intent_filters::android.intent.category.HOME
-3.80 intent_filters::android.intent.category.LAUNCHER
3.66 receivers::com.google.android.c2dm.C2DMBroadcastReceiver
3.39 req_permissions::android.permission.READ_PHONE_STATE
```

```
Explanations for sample 5 (true class: 0)
-6.06 suspicious_calls::android/net/Uri;->toString
5.14 req_permissions::android.permission.INTERNET
-4.96 used_permissions::ACCESS_NETWORK_STATE
-4.66 api_calls::android/media/MediaPlayer;->start
-3.89 used_permissions::VIBRATE
-3.35 intent_filters::android.intent.category.LAUNCHER
-3.15 used_permissions::ACCESS_FINE_LOCATION
3.02 suspicious_calls::java/lang/reflect/Method;->invoke
-2.89 api_calls::android/os/Vibrator;->vibrate
-2.44 suspicious_calls::android/net/Uri;->getLastPathSegment
```

```
18 res_list = list(filter(lambda x: ds.header.feats_desc[x] == "suspicious_calls::android/telephony/TelephonyManager;->getNetworkOperator", range(len(ds.head
19 print(res_list)

Explanations for sample 138 (true class: 1)
[449006]
```

```
predict CArray([0 0 0 0 0 0 0 0 0 0])
```

Next Plan

We have explored many features like how to data poison the model to reduce the accuracies. We need to select the best of them to poison the data (both evasion and poisoning) and see which attack is the most effective and try to identify these techniques to prevent these types of attacks.

Issues faced during the midterm phase

As the concepts were relatively new to everyone, it took some time to get the basics and get started on the work. Eventually this problem was solved, and everyone was on the right track with the same pace. We had few issues working with the existing data structures of secML and how to use them with sci-kit learn. We have researched well to overcome this problem. It was difficult to work with sparse matrices as it was hard to visually see the features.

Final Progress update

We first worked on generating adversarial examples to confuse SVM classifier (Data Evasion). There are already preexisting data poisoning methods available through the secML. But we decided to try our own unique methods using the gradient * input-based method. We took the most benign features we can find using the method (gradient * input) and then add the actual malicious feature we want to get by the classifier. We add the benign values to mask the actual malicious permission we want to get by the classifier and it was effective. This is assuming we own the testing data.

Evasion

Program Features

- 486013 = -10
- 738587 = -27
- 449006 = 60
- 334384 = -5
- 896875 = -23

Verdict: Benign

← Malicious

```
449006:"suspicious_calls::android/telephony/TelephonyManager;->getNetworkOperator"
```

* WE are on Testing Data

For data poisoning we would add a bunch of benign values into the training set whenever there is a malicious classification. Therefore, we will degrade the data that the model will train on and reduce the accuracy.

Poisoning

Training Set

- 486013 = -10
- 738587 = -27
- 449006 = 60
- 334384 = -5
- 896875 = -23
- {more benign values...}

Verdict: Benign

Ex: row 151 add benign features

```
449006:"suspicious_calls::android/telephony/TelephonyManager;->getNetworkOperator"
```

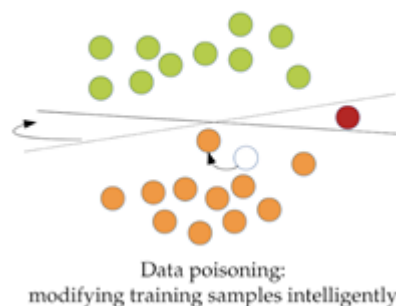
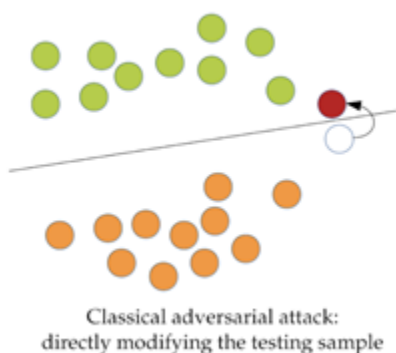
* WE are on Training Data

However this is easier to spot as model administrators can spot feature drift and reduced accuracy. Our group is currently making the process to where a user can select a specific permission they want to poisoning in the model and then automatically change the (gradient * input score) to make that said permission less malicious. This is more targeted and harder to spot. If we are able to make a malicious feature benign, then it is easier to slip by the model administrator. For example, if we can change the score from 60.5 to -12.5 then that feature will be classified as benign.

We realized that it is extremely hard to stop these types of targeted poisoning attacks. The best defense is to not allow bad actors access to the training data. If it is breached, another way is to actually have a deep understanding of the data and individual features. If one knows that certain features if perturbed can lead to drastic changes, then certain alarms can be implemented to detect changes. To mitigate damage done by poisoning attacks, there needs to be a continuous effort to actually understand what decisions the model is making and what data the model is using to make those decisions.

Lessons Learned

Attackers can target Machine Learning in any way by injecting the malware before training the data which is data poisoning so the resultant model itself will be faulty and classifies most of the data wrong, or injecting the malware in the test data which reduces the overall accuracy of the test data.



The above photos show how the data can be classified wrong according to the ability of the attacker to attack at which situation. Thus exceptional models/methods need to be developed such that the method needs to identify the weakest parameters or features in the dataset. Malware or the outliers which change the accuracy of the model needs to be identified at

different paths of developing the model. These anomalies must be identified such that the overall model can never be attacked by the attacker at any stage.

Future Work

As the technology develops the way to attack also may develop. So our resultant model also needs to adapt to the change such that it needs to identify these types of attacks in advance. Any loss in data or change in accuracy might be a small issue now but in the future where most of the decisions are made on the big data, these decisions must be accurate to the point or else might result in heavy losses.

Google Colaboratory Research Link

<https://colab.research.google.com/drive/1n9X3dCylhrbsMOa-uHrq-k8K03RohOzF?usp=sharing>

Literature Review (Previous Work)

Severi, Giorgio, et al. "Explanation-Guided Backdoor Poisoning Attacks against Malware

Classifiers." *ArXiv.org*, 11 Jan. 2021, <https://arxiv.org/abs/2003.01031>.

This paper goes through effective attacks on ML models specifically using PDFs, PeX files as inputs. This paper goes through effective ways on how to disguise malicious code in a format where the models are not able to easily recognize the features and think it is a harmless file. It also goes through ways of defending against such attacks and how feasible the defenses are.

Rosenberg, Ihai, et al. "Adversarial Machine Learning Attacks and Defense Methods in the Cyber Security Domain." *ArXiv.org*, 13 Mar. 2021, <https://arxiv.org/abs/2007.02407>.

This paper compiles the latest list of possible attacks against machine learning algorithms and possible defenses. For example it goes through blackbox based attacks for facial recognition and fingerprinting and also adversarial attacks to generate fake data.

Mehra, Akshay, et al. "How Robust Are Randomized Smoothing Based Defenses to Data Poisoning?" *ArXiv.org*, 2 Dec. 2020, <https://arxiv.org/abs/2012.01274v1>.

This paper specifically focuses on defense and goes through the randomized smoothing technique. Using this technique can make it a lot harder to poison data but it is not failure proof. They were able to poison the MNIST handwriting dataset with imperceptible changes.

All These works identified the possibility and threat of adversaries modifying the data behind Machine Learning models that identify malicious software. These works also identified the different ways that these models identify malicious software, as well as identifying the way that adversaries might attempt to affect the data behind the model.

However, our project attempts to test these ideas on real datasets of malicious software. Our project will clarify the real threat that adversaries can have by poisoning datasets that are in use by actors that identify malicious software.

Bibliography

- [1] [How Robust Are Randomized Smoothing Based Defenses to Data Poisoning? \(thevcf.com\)](https://thevcf.com)
- [2] [Adversarial Machine Learning Attacks and Defense Methods in the Cyber Security Domain | ACM Computing Surveys](#)
- [3] [Explanation-Guided Backdoor Poisoning Attacks Against Malware Classifiers | USENIX](#)

Biographical Sketches

Rishi Dandu- I am a graduate student at University of Texas at Dallas. I am currently majoring in Computer science and I have an interest in backend development and machine learning. Before coming to UT Dallas, I have experience at Amazon as a SWE intern.

Raviteja Avuthu - I have a bachelors and master's degree in Computer Science and am doing my second masters at The University of Texas At Dallas, graduating in December 2023 with a focus in Data Science. I have experience working in IBM as a Software Developer. I have done Machine Learning as an undergraduate course and am looking forward to gaining expertise in the field of Data Science. I also have experience in Python and R.

Harrison Hamric - I am an Undergraduate student in the Fast Track program at the University of Texas at Dallas. I am pursuing a Masters degree in Computer Science with a focus in Cybersecurity. I am a Scholarship for Service recipient, and plan to work for the US Federal Government, aiding in their cybersecurity defense mission. I am interested in system security, C\C++, and machine learning.

Payton Harmon - I am an Undergraduate student at the University of Texas at Dallas. I am currently enrolled in the Fast Track program here, and plan to pursue a Masters degree in Computer Science. I am a Scholarship for Service recipient.

Ashrafi Akbar - I am currently a PhD student at the University of Texas at Dallas. I have worked as a Junior software engineer for a couple years and am currently doing research in context aware learning and natural language processing.

